

Reducing the Progress-Diagonal Ridge

Coverage experiments and the use-point wall

local_mixing project

July 23, 2026

branch `ssg-gen-mix-clean` — tools `fmix --gen-snap-every` (49cee4c8) and
`src/bin/pairfloat_exp.rs` (5a9f449c)

Abstract

Local mixing rewrites a small circuit C into a large equivalent G that should hide C 's computation. The sharpest leak is the *progress diagonal*: a bright ridge in the affine-reconstruction heatmap between C and G , showing how far through the original computation any prefix of G has reached. We ask whether the ridge depth can be driven to 0.05 or below. After clearing two non-levers (generation dose and affine moves), we test a family of *free non-linear brackets* — inserted identity pairs floated apart — across four coverage strategies of increasing sophistication. All fail, the most careful one (gap-tiling) failing *backwards*: more masking sharpens the ridge. The failures are unanimous and their cause is a single structural fact, the *use-point theorem*: the ridge lives where operands are consumed, and no relocation-based masking touches that location. The only remaining route is computation on encoded operands (share-native / mask-aware gates).

1 The question

The reconstruction heatmap $H(i, j)$ is the best-effort GF(2)-affine reconstruction error of C 's state after i gates from *all* wires of G 's state after j gates. It shows a bright ridge along the line where G 's computational progress matches C 's; the ridge reveals the temporal location of the computation. We read the heatmap by the *ridge measure* (`reports/plot_hmap_ridge.py`): **depth** (ridge prominence, 0 = flat/ideal, ~ 0.5 = a maximally bright ridge), **rho** (Spearman of C -prefix vs. ridge column; **rho** = 1 is a clean monotone diagonal), a permutation z -score, and **contrast**. The ridge has survived every mixing method tried, always at **rho** = 1, with **depth** floored near 0.23–0.35. This note asks whether **depth** can reach 0.05.

2 Two non-levers

Generation dose. `fmix` phase A drives each gate through a target number of database re-encodings (“generation”). We ran a fresh gadgetized $n = 128$ sliced sandwich (85,058 gates) to generation 300 — six times the benchmarked dose — snapshotting at each generation multiple of 50 via the new `--gen-snap-every` flag (Fig. 1). The ridge only sharpens toward a floor:

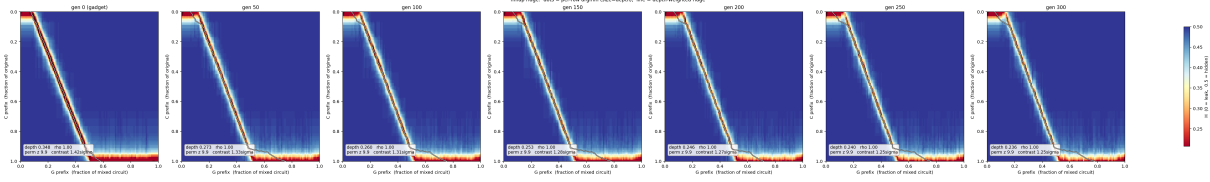


Figure 1: Generation-dose progression, $\text{gen } 0 \rightarrow 300$. The diagonal stays at $\text{rho} = 1$; only the depth shallows, toward a floor near 0.23.

generation	0	50	100	150	200	250	300
gates	85,058	179,437	218,809	265,096	317,289	377,912	450,318
ridge depth	0.348	0.273	0.261	0.254	0.246	0.240	0.236
rho	1.00	1.00	1.00	1.00	1.00	1.00	1.00

Decrements shrink geometrically toward $\sim 0.22\text{--}0.23$. Six times the dose past generation 50 bought a 14% depth reduction at $2.5\times$ the size.

Affine moves. The measure is invariant to the affine part of any encoding (CNOT, negation, XOR-sharing). Every conjugation twist in `fmix` is affine, so it *provably* cannot move the degree-1 ridge, at any dose. What is left is genuinely nonlinear re-encoding, which database replacements supply only inside ≤ 5 -gate windows; the 0.23 floor is the saturation of that thin nonlinear layer.

3 Free nonlinear brackets

Insert an identical pair $g \cdot g$ (the identity) into C , then float the two copies apart by adjacent swaps through gates they commute with. Correctness is exact and preserved bit-for-bit. Between the separated copies exactly one g is applied, so every prefix cut there reads the g -image of the true state — the target of g appears at degree $1 + (\# \text{controls})$. A separated pair is a *free nonlinear bracket*: no rewriting, only relocation. The tool `pairfloat_exp` builds a fixed C (3000 g57 gates, 128 wires) and tests families of this move, verifying equivalence on 200 random inputs per output.

4 Four attempts

4.1 Insertion count k (g57 pairs, commuting float).

k	0	1	2	3	4	5	8	11	16	24
depth	0.474	0.395	0.377	0.370	0.367	0.364	0.359	0.357	0.357	0.351

The first pair helps; the curve then flattens onto ~ 0.35 . *Self-dilution*: straddling brackets per cut = pairs $\times$ span/size = $84000/(1+2k)$, saturating at $\text{span}/2 \approx 42$ ($\sim 27\%$ of wires) as $k \rightarrow \infty$ — each pair inflates the circuit it must cover. A model $\text{depth} \approx \text{depth}_0 (1 - \text{coverage})$ fits within ~ 0.01 .

4.2 Cross budget B (float past colliders by conjugation). Free floating stops after ~ 42 slots. Crossing up to B colliders via the exact splitting rules (`postmix::rules R1/R2/R3`)

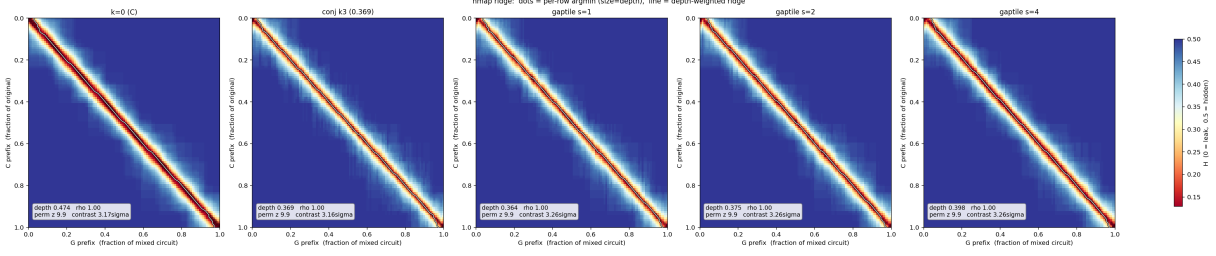


Figure 2: Gap-tiling $s = 1, 2, 4$ (rightmost three) against C -vs- C and the random-insertion baseline. Depth *rises* with stacking: masking between uses cannot reach the diagonal.

grows span, size, and loses half the floaters as colliders:

B (at $k = 3$)	0	1	2	4	8
depth	0.369	0.343	0.333	0.330	0.329
mean travel	41	120	192	302	405
gates	21k	53k	77k	105k	129k

Span rose $10\times$; depth saturated at 0.33; the whole gain is banked by $B = 1$. Crossing is where the free bracket becomes priced, and the price outruns the benefit at once.

4.3 Adaptive wire selection. Choosing each pair’s wires for maximal mobility (quiet target, unwritten controls) raised span $7.5\text{--}50\times$ and made depth *worse*:

adaptive	$k=1$	$k=3$	$k=5$	$+B=4$	$+B=8$
depth	0.437	0.427	0.423	0.402	0.396
mean travel	150	310	432	1486	2039

Target concentration: a wire is mobile because nothing near it is active, so it stays quiet for hundreds of cuts and every insertion samples the same tiny pool. Distinct masked wires collapse to $\sim 10\text{--}20$ of 128, dropping coverage below random’s. Mobility and diversity are in tension.

4.4 Gap-tiling — the decisive result. Invert the assignment: for each wire w , every maximal stretch of C between consecutive touches of w is a bracket slot; insert s distinct pairs targeting w there (controls untargeted in the gap, so the pair commutes with the interior), floated to the gap ends. This forces diversity over all wires and masks every wire at every cut *except its own use points*. Material cost matched prediction ($\times 6.9/12.9/24.8$ for $s = 1/2/4$); the predicted depth was $0.25/0.12/0.05$. The result was the opposite (Fig. 2):

gap-tiling s	1	2	4
depth	0.364	0.375	0.398
contrast	3.26	3.26	3.26

Stacking masks makes the ridge *sharper*. The plate shows why: the off-diagonal background hides more uniformly while the diagonal is untouched and stands out more by contrast.

5 Conclusion: the use-point wall

Theorem 1 (use-point, empirical). *Between-use bracket masking — in any form (random k , cross budget, adaptive mobility, gap-tiling) — cannot reduce the degree-1 progress ridge below ~ 0.33 . It targets the wrong location.*

Gap-tiling masks each wire only strictly between its uses (floaters stop at the touch gates), but the ridge at C -row i aligns with the G -column where gate i 's operands are *used*, and those are left exposed. The diagonal *is* the operand-use locus, so between-use masking cannot reach it; hiding the idle background only sharpens the aligned column by contrast. The small reductions the earlier modes obtained ($0.47 \rightarrow 0.33$) came precisely from brackets that *accidentally straddled* use points while floating blindly — gap-tiling removes those straddles and does *worse* than random.

This is the same wall the deferred-mask work (`docs/NONLINEAR_SHARE_ENCODING`) hit from the other side, where a peek un-masked each operand at its use point and re-exposed the diagonal. Two independent attacks converge: the ridge cannot be moved by relocation.

6 What remains

To reduce the ridge one must mask a wire *at the instant it is consumed*: compute a gate on the encoded operand $w \oplus \text{mask}$ without ever reconstructing w . That is share-native / mask-aware computation — what the legacy g57 gadget does (it computes on shares and never reconstructs an operand, which is why it hides at degree 2). The open construction is a mask-aware CG that folds the mask correction into the gate; its cost is the answer-extraction interface, not gate count. Coverage-only approaches, and their $\sim \times 25$ -material price for full tiling, are a closed dead end.